

Brendon Orchards Database

...

By Aryan, Charles, Emilia and Tom

Our Client

Brendon Orchards Cooperative are:

- A Cooperative that rent out equipment for turning spare apples into apple juice or cider.
- Based in Somerset.
- Comprised of a small committee alongside members who volunteer to perform different roles.

Due to this they have very limited technical skills and knowledge.

Project Overview

They have asked us to:

- Redesign their database
- Move it onto a more user friendly system
- Add automation between the website and the database.

This will save them time and help them comply with GDPR, while also improving the experience for members.

User stories (Admin)

As an admin at Brendon Orchards, I would like:

- A multi-user database that prevents clashes
- To quickly access booking history and membership numbers
- Automated updates after bookings so I don't have to manually enter data or generate invoices
- A system that is easy to use and doesn't require me to learn difficult new skills.

User stories (Customer)

As a Customer, I would like:

- Easy registration & membership renewal
- To check equipment availability easily with a visual representation
- A smooth, clear experience that lets me book without admin intervention
- A satisfying and safe user experience.

What do Brendon Orchards currently use?

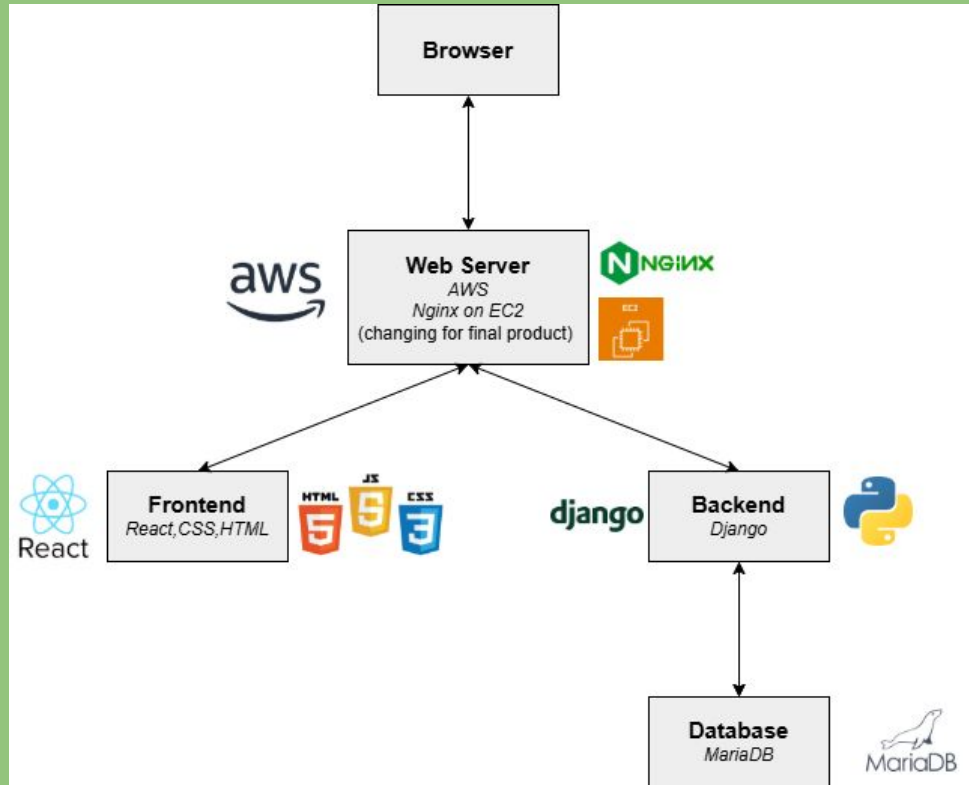
- Large Access file in Dropbox
- Not connected to the WordPress booking website
- Prone to conflicts when multiple admins open/edit it
- Manual data entry for every booking and membership
- GDPR concern: admins storing local copies

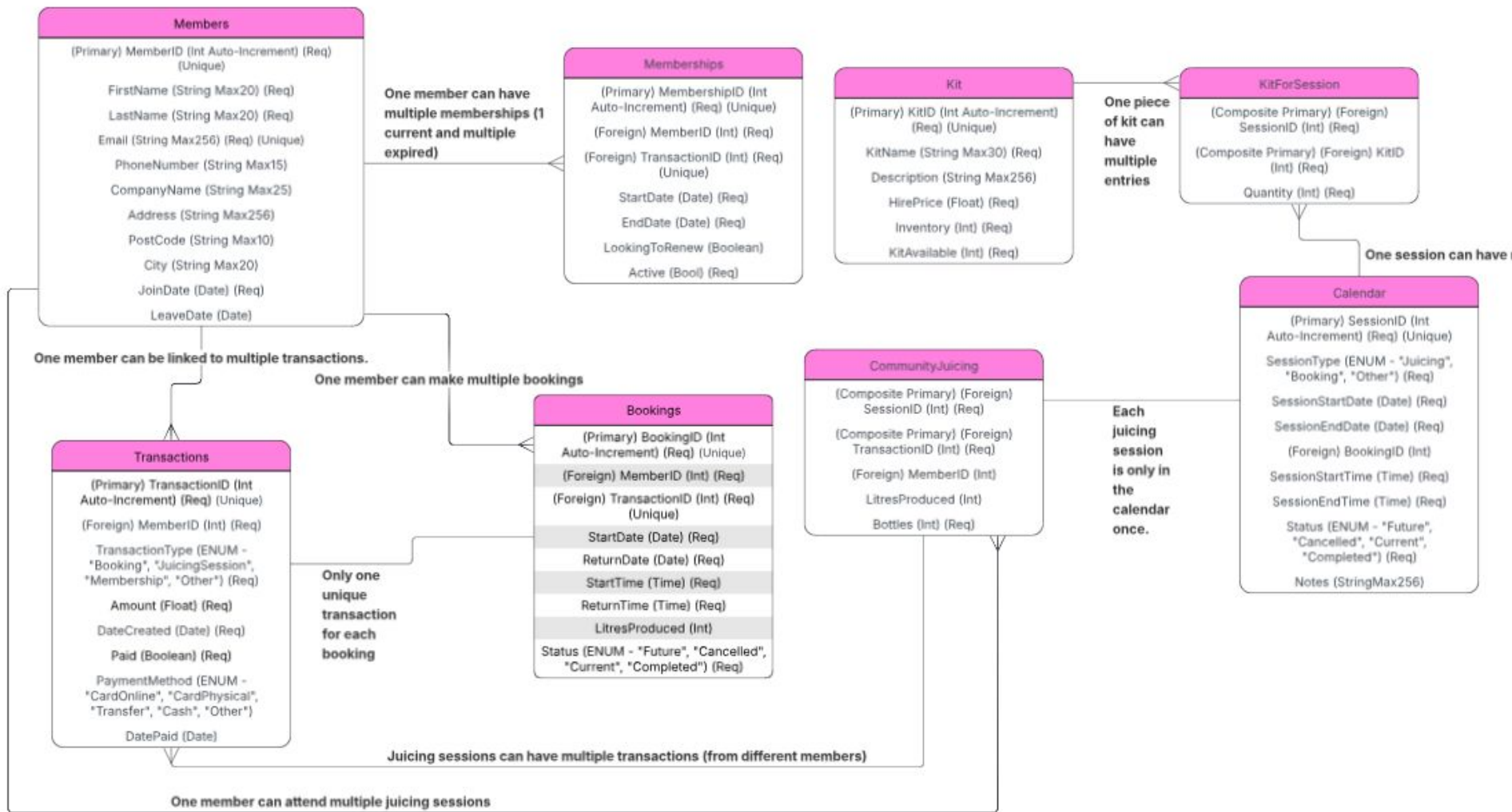
Tech Stack + Architecture

We are using django to create a web application that connects to a mariadb database.

- Backend: Django REST framework
- Database: MariaDB
- Frontend: React
- Hosting (for Beta): AWS for Beta, Client's current host for final product.
- Why?
 - Django : built in admin panel, strong foundation to build a secure and structured backend.
 - MariaDB : stable open-source SQL database compatible with client.
 - React : clean and responsive UI for membership (both for admins & members) + booking portals

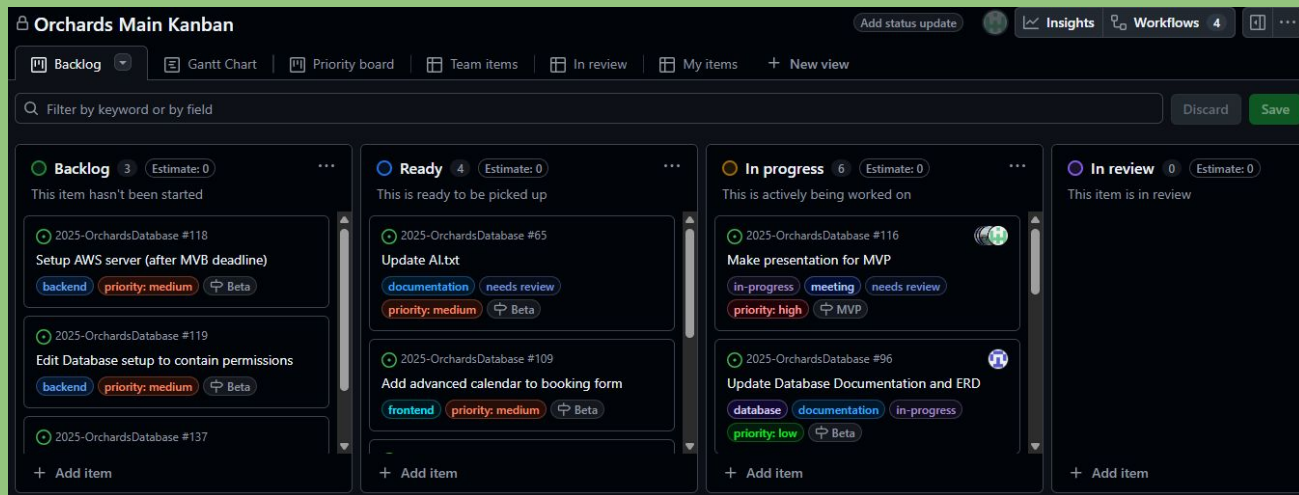
Tech Stack + Architecture





Project Organisation

- We used GitHub Projects to track issues & PRs.
- Used Discussions/PR reviews to coordinate backend–frontend integration
- Followed small-task commits to solve issues.
- Initially we didn't have a designated project manager, however in the build up to our MVP we realised the value in this and have started rotating every three weeks.



Live Demo

Issues Faced & Solutions

Issue: Django & MariaDB connection failures. migrations not applying or not updating properly.

Solution: Standardised MariaDB install, fixed user permissions, rebuilt migrations with a clean schema. Will be easier to handle as soon as DB is shared on host.

Issue: Confusion with routing structure (api/me, account urls, etc.).

Solution: Reorganised Django URLs, separated bookings vs account apps cleanly to match REST conventions.

Issue: Register/login not updating frontend state.

Solution: Implemented CSRF cookies, Axios config, tested via Post API & DB .

Issue: Members not showing in /members/get-all.

Solution: Rebuilt model relationships, validated serializers, and fixed inconsistent test data.

Issues Faced & Solutions

Issue: Strict budget and limited ability to spend on web hosting.

Solution: We are developing using AWS however before our final release we will be moving to a private host decided upon by the client who is at a price point they are happy with. This also means we need to be more thoughtful and efficient with our design of the application and database.

Issue: Small features/changes to the database requested by the client.

Solution: Django is a flexible framework that is allowing us to make small changes to our database design as we go. This means we can integrate client feedback continuously.

Issue: Client uncertainty over maintenance and development after the project ends.

Solution: Reassurance that we will leave them with all of the documentation and knowledge to use and maintain the product going forwards.

Information About Client Demonstration

Since our client's technical skills are so limited, we handled our MVP release slightly differently:

- No early build release to avoid confusion.
- Instead we held a live demo.
- We made sure to ask open-ended questions throughout to gain valuable feedback.

Client Feedback

Alongside our MVP release meeting, we have been gathering feedback through biweekly meetings with our client.

Key feedback:

- Growing trust over time
- Positive response to foundations
- Concerns over frontend UI
- Questions over domain trust
- Simplified admin UI panel
- Support after end of project
- GDPR issues regarding DB data

Future Plans

Based on our initial roadmap, MVP progress and client feedback these are our upcoming goals and future plans:

Beta:

- Simplified admin UI
- Initial branded UI for frontend (CSS)
- Complete booking & backend integration
- Launch on AWS hosting
- Finalise DB & start using a shared DB
- Permission-locking APIs

Final product:

- Documentation + video walkthroughs (maintenance and features)
- GDPR & data retention plans
- Data migration from Access to MariaDB
- Launch on client's hosting
- Extra backend features (invoice, emails etc.)

Thank you for listening.
Any questions?

Staging endpoint (only good for developers work) endpoints (admin,api,bookings)

